



UNIVERSITAS
GADJAH MADA

CuciGo

Software Engi- neering Process

Group 6 | Universitas Gadjah Mada (UGM)



Contents

Project Overview
AI-Assisted Software Engineering Process
Phase 1: Ideation & SRS Creation
Phase 2: UI/UX Design
Phase 3: Software Development & Implementation

Testing & Quality Assurance
Demo & Results
AI Tools Comparison
Lessons Learned
Conclusion & Future Work



What is CuciGo?

CuciGo is a mobile-first laundry pickup and delivery marketplace connecting three user groups:

Students

Browse services,
place orders, track
laundry, manage
payments

Partners

Manage incoming
orders, update
status, set pricing

Drivers

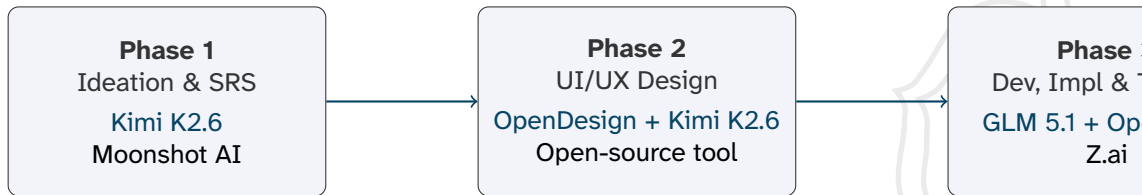
Navigate routes,
confirm pickups,
update delivery
status

Key Facts

- 30 functional requirements (FR-1 to FR-30)
- 4 user roles: Student, Partner, Driver, Admin
- Tech stack: Next.js, TypeScript, Tailwind CSS, Prisma + SQLite

Three-Phase AI-Assisted SE Process

Our project followed a **three-phase software engineering process**, each phase leveraging a different AI tool:



Why AI-Assisted?

Each AI model was chosen for its strengths: Kimi for research and documentation, OpenDesign for visual prototyping, and GLM 5.1 for code generation with the OpenCode agentic harness.

Phase 1 — Ideation & SRS Creation

Tool: Kimi K2.6 (Moonshot AI)

Process:

1. Problem identification — laundry pain points for Anak Kos
2. Feature brainstorming with multi-turn dialogue
3. Requirement elicitation following IEEE 830 structure
4. SRS document drafting with functional and non-functional requirements
5. Iterative refinement and compliance verification (PDP Law, Bank Indonesia)

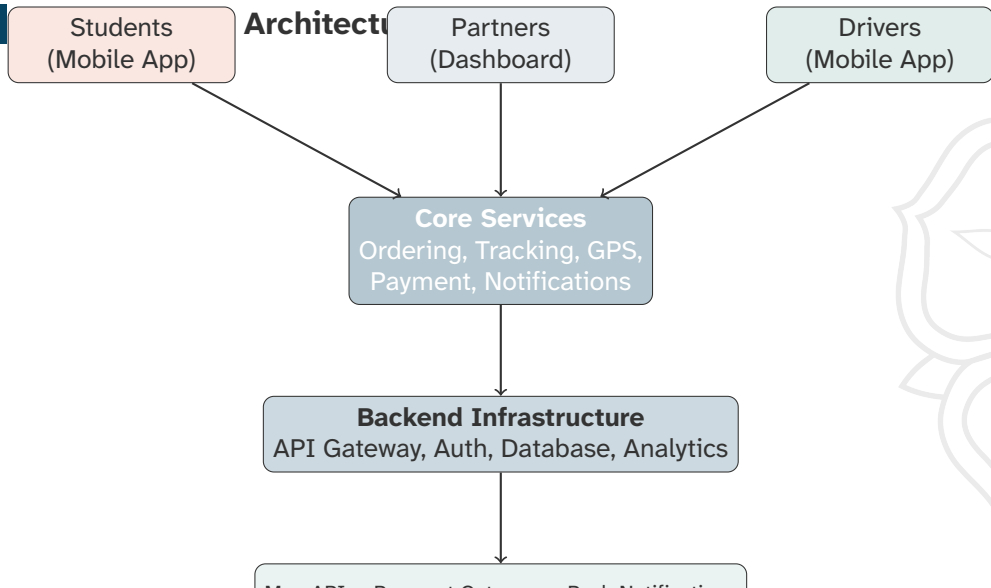
SRS Outputs

- 30 functional requirements (FR-1 to FR-30)
- 17 quality requirements (QR-1 to QR-17)
- 4 compliance requirements (CR-1 to CR-4)
- 4 user role definitions
- System architecture diagram
- Verification plan

SRS — Functional Requirements Overview

Category	FR Range	Key Features
Authentication	FR-1 to FR-5	Register, login, RBAC, profile
Laundry Ordering	FR-6 to FR-10	Service selection, weight estimation, scheduling
Real-Time Tracking	FR-11 to FR-14	Status updates, push notifications, map
GPS Location	FR-15 to FR-18	Auto-detection, manual correction, routing
Payment & Finance	FR-19 to FR-23	E-wallet, bank transfer, spending tracker
Student Utilities	FR-24 to FR-26	Shared laundry, reminders, favorites
Admin System	FR-27 to FR-30	Dashboards, analytics, user management

Architecture



Phase 2 – UI/UX Design

Tool: OpenDesign (open-source) + Kimi K2.6

Process:

1. Defined design tokens from initial mood boarding
2. Created CSS custom properties (oklch color system)
3. Generated 13 HTML screen prototypes:
 - 10 student screens
 - 1 driver screen
 - 1 partner screen
 - 1 admin screen
4. Iterative feedback with Kimi to refine layouts

Design System

- **Colors:** Oklch-based tokens (primary teal, secondary coral)
- **Typography:** System font stack
- **Mobile:** 390×844px frame
- **Spacing:** 4px scale system
- **Components:** Buttons, cards, chips, inputs, tab bars

UI/UX — Screen Inventory

Role	Screen	Purpose
Student	Splash	Onboarding & feature highlights
Student	Login	Phone/email auth with OTP
Student	Home	Services, orders, nearby laundry
Student	New Order	Service, weight, schedule, location
Student	Order Summary	Confirmation with timeline
Student	Payment	E-wallet/bank selection
Student	Tracking	Real-time map & status
Student	Orders	History with filter tabs
Student	Shared Laundry	Roommate cost-splitting
Student	Spending	Monthly charts & breakdown
Driver	Home	Active deliveries & navigation
Partner	Dashboard	Order management & analytics
Admin	Dashboard	System health & user management

Phase 3 — Development & Implementation

Tool: GEM 5.1 via OpenCode Harness

Process:

1. Analyzed HTML prototypes and SRS document
2. Set up Next.js 16 project with TypeScript, Tailwind CSS v4
3. Designed database schema (Prisma + SQLite)
4. Implemented 14 route pages translating mockups to React components
5. Created shared UI component library
6. Built server actions for auth and order management
7. Seeded database with realistic test data

Tech Stack

- **Framework:** Next.js 16 (App Router)
- **Language:** TypeScript
- **Styling:** Tailwind CSS v4
- **Database:** SQLite via Prisma 7
- **Auth:** Cookie-based sessions, bcryptjs
- **Design:** Oklch color tokens, mobile-first



Database Schema

Core Models

- **User** — id, name, email, phone, role, password
- **Address** — id, userId, label, address, lat/lng
- **LaundryPartner** — id, userId, name, rating, prices
- **PartnerPrice** — id, partnerId, serviceType, pricePerKg
- **Order** — id, orderCode, studentId, partnerId, driverId, serviceType, weight, status,

Supporting Models

- **OrderStatusHistory** — id, orderId, status, note, timestamp
- **SharedGroup** — id, name, adminId
- **SharedGroupMember** — id, groupId, userId, shareAmount, paymentStatus
- **SharedOrder** — id, orderId, groupId, totalSplit
- **Notification** — id, userId, or-

Project Structure

```
1 cucigo/
2 +-- prisma/
3 |   +-- schema.prisma      # Database schema (11 models)
4 |   +-- seed.ts            # Sample data seeding script
5 |   +-- dev.db             # SQLite database
6 +-- src/
7 |   +-- actions/auth.ts    # Server actions (login, order, status)
8 |   +-- app/
9 |   |   +-- page.tsx       # Splash/onboarding
10 |   |   +-- auth/login/    # Authentication
11 |   |   +-- student/       # 8 student routes
12 |   |   +-- driver/home/   # Driver dashboard
13 |   |   +-- partner/       # Partner dashboard
14 |   |   +-- admin/         # Admin dashboard
15 |   +-- components/
16 |   |   +-- ui/            # Button, Card, Chip, Input, StatusDot
17 |   |   +-- layout/        # MobileFrame, NavBar, TabBar
18 |   +-- lib/
19 |       +-- auth.ts        # Session management, password hashing
20 |       +-- constants.ts   # Service types, order statuses, helpers
```



Testing & Quality Assurance

Build & Type Safety

- All 14 routes compile and build successfully
- TypeScript strict mode enabled
- Prisma generates type-safe database client
- Server actions provide type-safe mutations
- ESLint configuration from Next.js defaults

Verification Strategy (from SRS)

- **Unit Testing** — Individual component testing
- **Integration Testing** — Module and external service interfaces
- **System Testing** — Full app against FR and QR
- **UAT** — Representatives from each user class validate in production-like environment



SRS Verification Traceability

Requirement	Method	Acceptance
FR-1 to FR-5	Unit & integration	Login/profile works
FR-6 to FR-10	System testing, UAT	Orders with correct pricing
FR-11 to FR-14	System testing, UAT	Real-time status updates
FR-15 to FR-18	System & field testing	GPS accuracy < 10m
FR-19 to FR-23	Integration with payment providers	Transactions recorded
FR-24 to FR-26	System testing, UAT	Shared laundry & reminders
FR-27 to FR-30	System testing, UAT	Admin dashboard correct data



UNIVERSITAS
GADJAH MADA

Application Demo



Application Routes Overview

Student Routes

- / — Splash / Onboarding
- /auth/login — Authentication
- /student/home — Dashboard
- /student/order — New order flow
- /student/order/[id]/summary — Order summary
- /student/order/[id]/payment

Student Routes (cont.)

- /student/shared — Shared laundry groups
- /student/spending — Spending tracker

Other Roles

- /driver/home — Driver dashboard
- /partner/dashboard — Partner dashboard
- /admin/dashboard —

AI Tools Comparison

Aspect	Kimi K2.6	OpenDesign + Kimi	GLM 5.1 + OpenCode
Phase	Ideation	UI/UX Design	Development
Task	SRS creation	Visual prototyping	Code implementation
Output	Requirements doc	13 HTML screens	Next.js app
Human Role	Prompt, review	Guide, iterate	Architect, verify
Strength	Long-context reasoning	Rapid visual design	Multi-file code generation
Limitation	No code execution	Static prototypes	Needs architectural guidance

Key Insight

Each AI tool was leveraged for its **comparative advantage**. Kimi excels at long-document generation and structured reasoning. OpenDesign provides rapid visual iteration. GLM 5.1 with OpenCode's agentic harness enables multi-file code generation with tool use capabilities (file read/write, shell commands, search).

Lessons Learned

What Worked Well

- AI-assisted ideation rapidly produced a comprehensive SRS document following IEEE 830 standards
- HTML prototypes enabled **visual stakeholder feedback** before writing any code
- OpenCode's agentic harness allowed parallel file creation, dramatically reducing development time
- Existing design tokens (CSS

Challenges

- Prisma v7 required driver adapters (breaking change from v6 patterns) — required resolving import paths and API changes
- AI-generated code needed **architectural review**: the agent sometimes chose suboptimal patterns that required manual correction
- Tailwind CSS v4 uses a different configuration approach (@theme

Conclusion & Future Work

Summary

- Successfully implemented a **full-stack CuciGo web application** covering all 4 user roles
- **14 routes** translating 13 HTML prototypes into functional React pages
- **11 database models** with seed data for testing
- Demonstrated a **three-phase AI-assisted SE process** using Kimi K2.6, OpenDesign, and GLM 5.1
- All pages pass **TypeScript strict mode** and **Next.js build verification**

Future Work

- **Phase 2 features:** Spending tracker analytics, shared laundry ordering flow

References

-  IEEE Standard 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE, 1998.
-  ISO/IEC/IEEE 29148:2018, *Systems and software engineering — Life cycle processes — Requirements engineering*, ISO, 2018.
-  Vercel, *Next.js Documentation*, <https://nextjs.org/docs>, 2026.
-  Prisma, *Prisma Documentation*, <https://www.prisma.io/docs>, 2026.
-  Tailwind Labs, *Tailwind CSS v4 Documentation*, <https://tailwindcss.com/docs>, 2026.
-  Moonshot AI, *Kimi K2.6*, <https://kimi.moonshot.cn>, 2026.
-  Z.ai, *GLM 5.1, OpenCode Agentic Harness*, 2026.
-  OpenDesign, *Open-source UI/UX Design Tool*, 2026.



UNIVERSITAS
GADJAH MADA

CuciGo

Software Engi- neering Process

Group 6 | Universitas Gadjah Mada (UGM)

